

|                           |                                                                                       |
|---------------------------|---------------------------------------------------------------------------------------|
| ДИСЦИПЛИНА                | <b>Разработка обеспечивающих подсистем</b>                                            |
|                           | (полное наименование дисциплины без сокращений)                                       |
| ИНСТИТУТ                  | <b>информационных технологий</b>                                                      |
| КАФЕДРА                   | <b>квантовых информационных технологий,<br/>практической и прикладной информатики</b> |
|                           | (полное наименование кафедры)                                                         |
| ВИД УЧЕБНОГО<br>МАТЕРИАЛА | <b>Материалы для практических/семинарских занятий</b>                                 |
|                           | (в соответствии с пп.1-11)                                                            |
| ПРЕПОДАВАТЕЛЬ             | <b>Кириллина Юлия Владимировна, Трушин Степан<br/>Михайлович</b>                      |
|                           | (фамилия, имя, отчество)                                                              |
| СЕМЕСТР                   | <b>7, 2026-2027</b>                                                                   |
|                           | (указать семестр обучения, учебный год)                                               |

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 8

**Цель занятия:** смоделировать взаимодействие компонентов системы и внешних сервисов, а также определить меры информационной безопасности для этих взаимодействий без разработки собственной спецификации API.

### Постановка задачи:

1. Определить интеграционные сценарии выбранной системы (например: аутентификация через внешнего IdP, оформление заказа с обращением к платёжному сервису, получение статуса доставки). Кратко описать участников и границы (внутренняя система, внешние сервисы).
2. Разработать UML Sequence Diagram для двух ключевых сценариев интеграции, указав сообщения, протоколы/форматы (на уровне «HTTP/HTTPS, JSON/XML»), возможные ошибки и ветвления.
3. Определить схему разграничения прав доступа пользователей (роли, права, ограничения).
4. Сформировать матрицу разграничения доступа (RBAC): роли × ресурсы × операции (Create/Read/Update/Delete/Execute).
5. Сформировать перечень мер информационной безопасности (аутентификация, авторизация, защита данных в канале, шифрование).
6. Использовать инструменты (Draw.io, текстовый редактор) для проектирования и визуализации схем.

### Теоретический вспомогательный материал

UML Sequence Diagram (диаграмма последовательности) — модель, описывающая обмен сообщениями между участниками системы во времени. Используется для уточнения сценариев взаимодействия: кто с кем общается, в каком порядке и при каких условиях.

#### Основные элементы:

1. Участники (Lifelines) — акторы, интерфейсы, компоненты,

внешние сервисы. Обозначаются прямоугольником с именем и пунктирной «жизненной линией».

2. Активация (Activation bar) — вертикальный прямоугольник на lifeline, показывающий период выполнения операции.
3. Сообщения (Messages):
  - синхронное вызова — сплошная стрелка с треугольной головкой; отправитель ждёт ответ;
  - асинхронное — сплошная стрелка с открытой головкой; отправитель не блокируется;
  - возврат — пунктирная стрелка (опционально);
  - самовывоз — стрелка на собственную lifeline;
  - создание/уничтожение объекта — сообщение «create» к новой lifeline / крестик «X» для завершения.
4. Комбинированные фрагменты (ограничители с ключевым словом):
  - alt — альтернативы с охранными условиями [условие].
  - opt — необязательная ветка.
  - loop — цикл с границами (например,  $i=1..N$ , [while ...]).
  - par — параллельные ветви.
  - break — прерывание сценария.
  - ref — ссылка на другой сценарий.
  - critical — критическая секция.

Пример Sequence Diagram представлен на Рисунке 4.

Разграничение прав пользователя следует оформить в виде Таблицы 13.

Таблица 13 — Разграничение прав пользователей

| Роль          | Доступные операции                                                  | Ограничения                             |
|---------------|---------------------------------------------------------------------|-----------------------------------------|
| Администратор | Управление пользователями, настройка системы, доступ ко всем данным | —                                       |
| Менеджер      | Создание/редактирование заказов, доступ к отчетам                   | Нет доступа к управлению пользователями |
| Клиент        | Создание заказов, просмотр статуса                                  | Нет доступа к чужим заказам             |

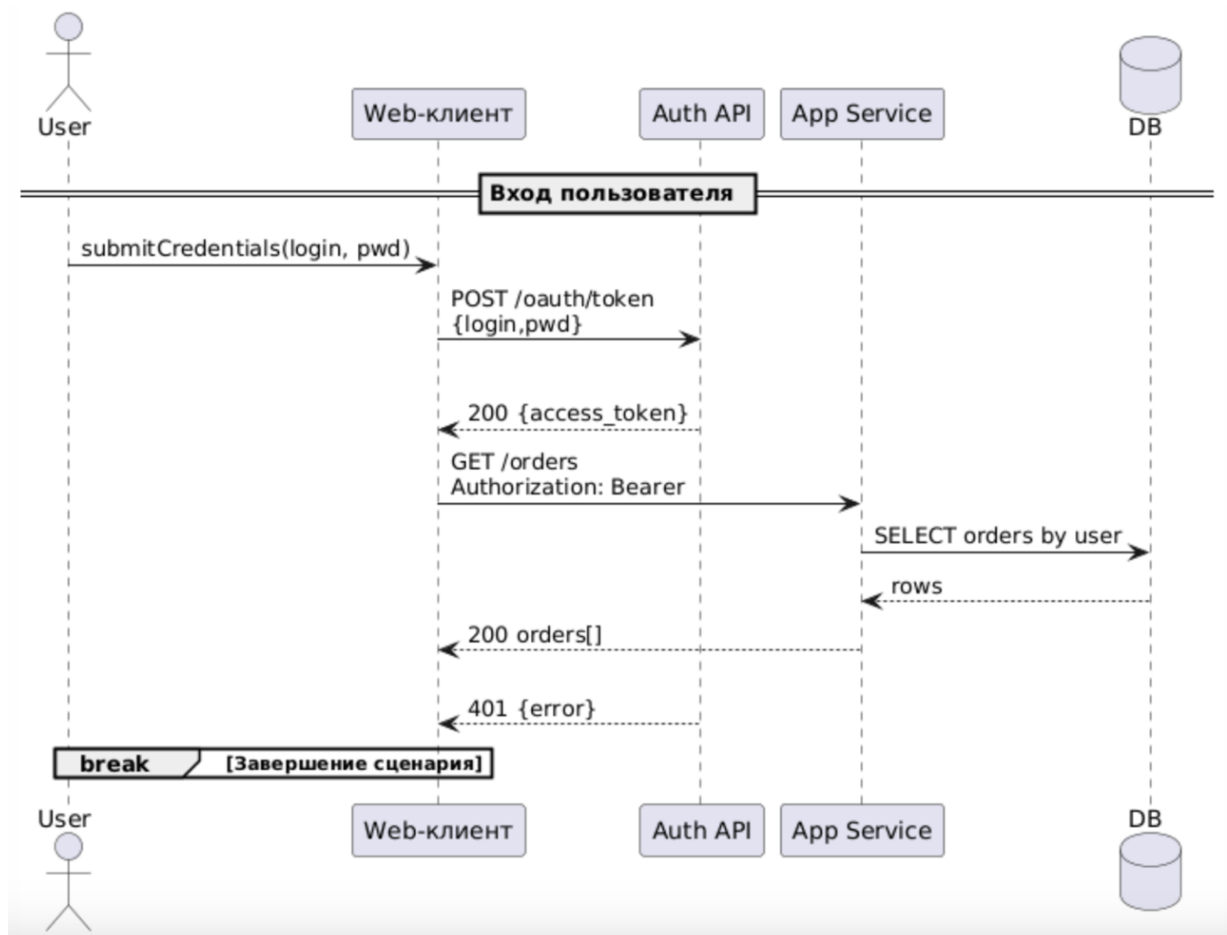


Рисунок 4 — Sequence Diagram

RBAC (Role-Based Access Control) — модель управления доступом, в которой права назначаются не пользователям напрямую, а ролям. Пользователь получает доступ, становясь участником одной или нескольких ролей. Базовая матрица фиксирует соответствие: Роль × Ресурс × Операция (CRUDX), где операции: Create/Read/Update/Delete/Execute.

Ключевые понятия:

1. Роль — набор разрешений, отражающий служебные обязанности (например, Пользователь, Менеджер, Администратор).
2. Ресурс — объект доступа (сущность системы, экран, API-эндпоинт, отчёт, очередь сообщений).
3. Операция — допустимое действие над ресурсом: С (создать), R (просмотреть), U (изменить), D (удалить), X (выполнить/запуск).
4. Ограничение — условие на уровень строки/поля/контекста (например, «только свои записи», «только в пределах

подразделения»).

5. Принцип наименьших привилегий — по умолчанию запрет, разрешается только необходимый минимум.

Виды ограничений:

1. Объектный уровень: доступ к конкретным ресурсам/экрану/эндпоинту.
2. Строковый уровень (Row-Level): фильтры по владельцу, подразделению, организации/тенанту.
3. Полевой уровень: запрет/только чтение отдельных атрибутов (например, скрыть зарплату).
4. Временные/контекстные: доступ в рабочее время, из доверенной сети и т. п.
5. Разделение обязанностей (SoD): несовместимые роли не должны назначаться одному пользователю (например, Создает платёж  $\neq$  Утверждает платёж).

Порядок разработки RBAC:

1. Определить ресурсы (сущности, отчёты, функции, эндпоинты).
2. Сформировать операции для каждого ресурса (CRUDX) и их смысл.
3. Описать роли (по бизнес-функциям), при необходимости ввести иерархию ролей.
4. Заполнить матрицу ролей и операций; для нестандартных случаев указать ограничения/фильтры.
5. Связать матрицу с механизмом исполнения:
  - в приложении — проверки на экранах/кнопках и при вызове сервисов;
  - в API — политики на эндпоинтах (scope/claim  $\rightarrow$  разрешения);
  - в БД — роли/гранты, политики RLS (Row-Level Security).
6. Определить процесс изменения (версионирование матрицы, утверждение ИБ, аудит).

Пример оформления RBAC представлен в Таблице 8.2

Таблица 14 — Пример матрицы RBAC

| Роль / Ресурс | Заказы              | Клиенты              | Отчёты | Настройки |
|---------------|---------------------|----------------------|--------|-----------|
| Гость         | R                   | —                    | —      | —         |
| Пользователь  | C/R/U (только свои) | R (своя организация) | —      | —         |
| Менеджер      | C/R/U               | C/R/U                | R/X    | —         |
| Администратор | C/R/U/D             | C/R/U/D              | R/X    | C/R/U/D   |

**Результат практического занятия:** отчет по восьмому практическому занятию в формате doc/docx (форма отчета размещена в СДО) и презентация в формате ppt/pptx.

Результат каждого практического занятия по дисциплине должен заноситься в единый отчет и в течение семестра размещаться в СДО в отведенную область для загрузки. Для отражения схем и таблиц можно использовать альбомный разворот листа.

В СДО загружается как отчет, так и презентация. В случае, если занятие предполагает публичную защиту, проводится обсуждение полученного результата.